

Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

University of Minho
School of Engineering

Data Security

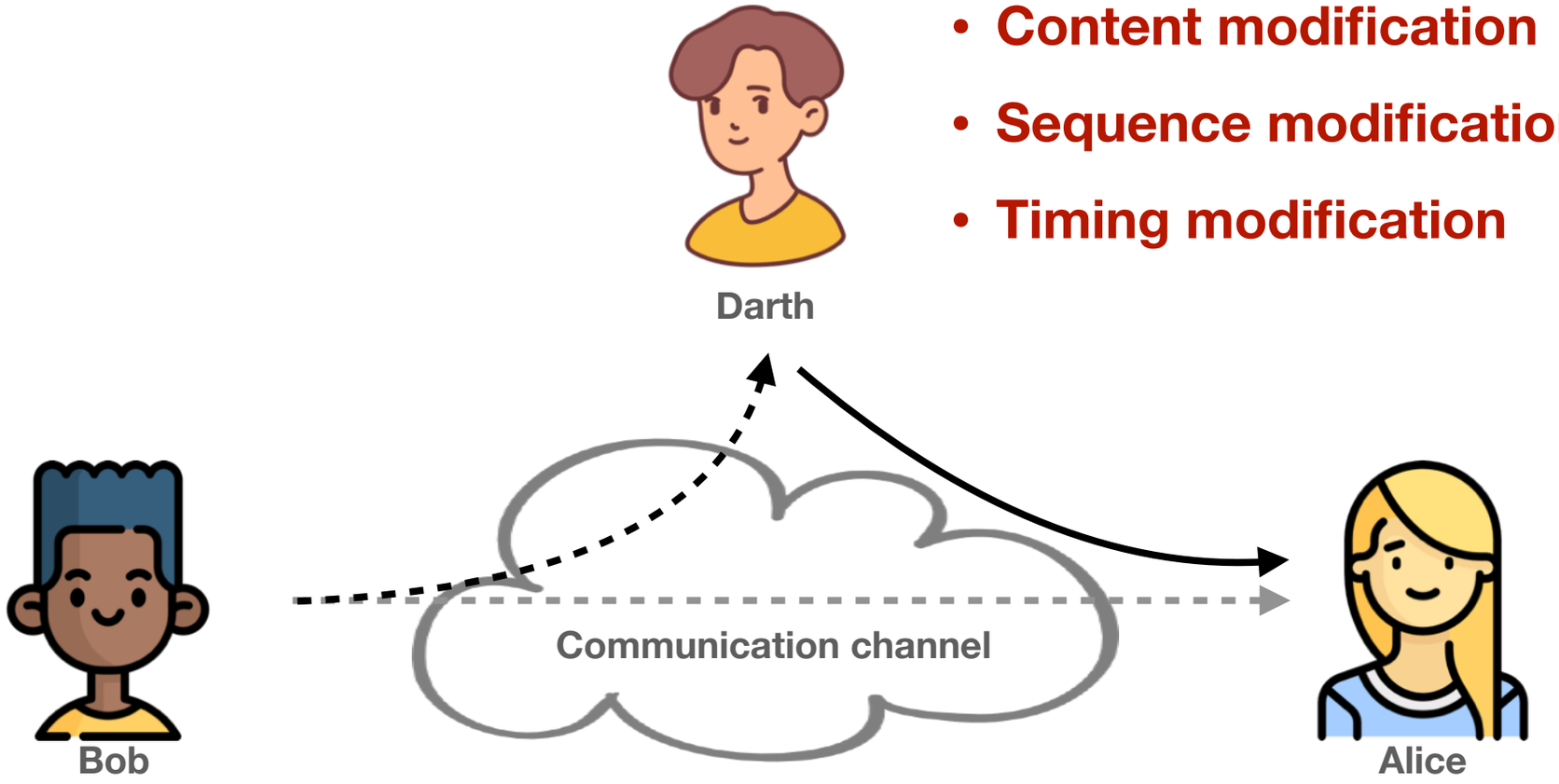
Objectives

- Comprehend the threats to the integrity of encrypted data
- Understand the main message authentication approaches
- Understand the main MAC schemes
- Understand the keyed-hashing message authentication strategy
- Understand the main authenticated encryption systems

Recap

Threats

- Insertion of messages
- Content modification
- Sequence modification
- Timing modification



Generally addressed through Message Authentication

Data Integrity

Message Authentication

Message authentication is a mechanism to verify that a message comes from the alleged source and has not been altered.

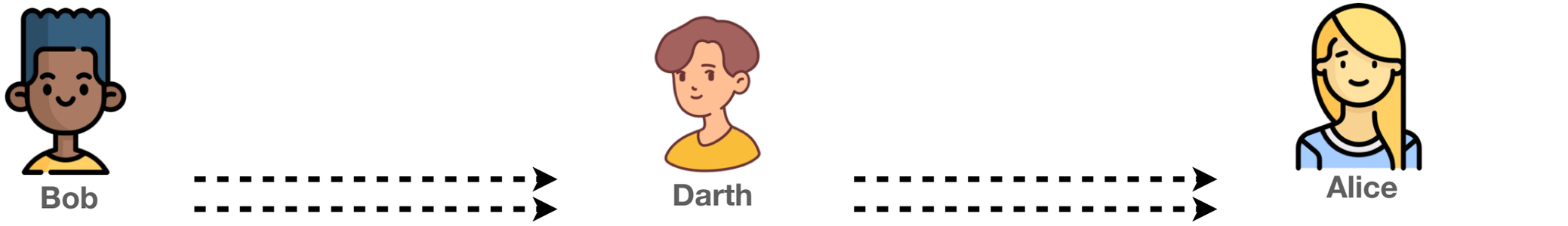
Source authenticity + Integrity

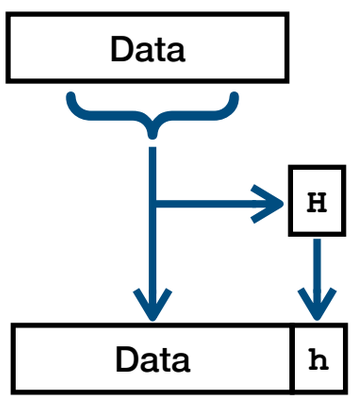
- The types of functions typically used:
  - Cryptographic hash functions
  - Message encryption
  - Message Authentication Code (MAC)

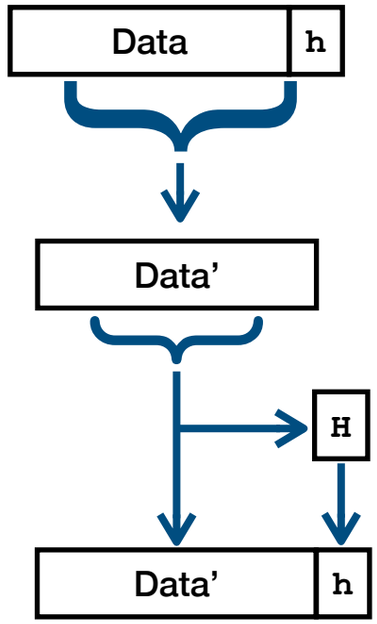
Data Integrity

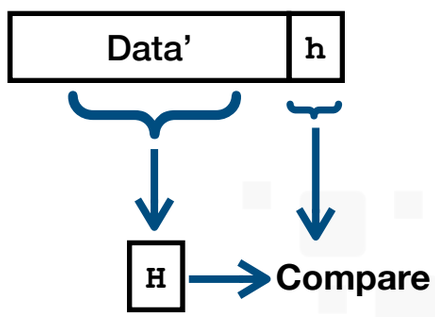
Message Authentication

How to provide message authentication using hash functions?







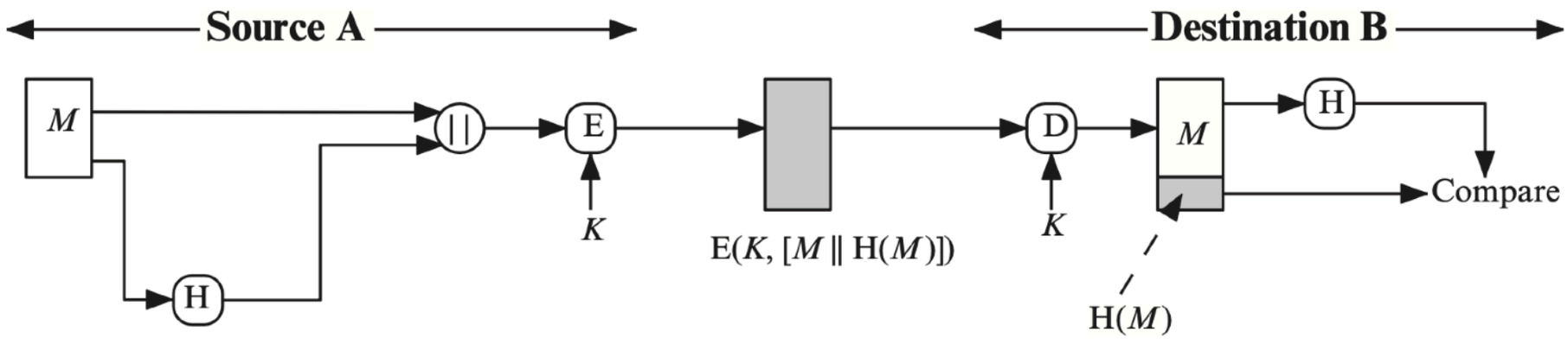


Man-in-the-middle attack

Data Integrity

Message Authentication

Using hash code and symmetric encryption for message authentication



Since K is shared only between A and B, the message origin must be A

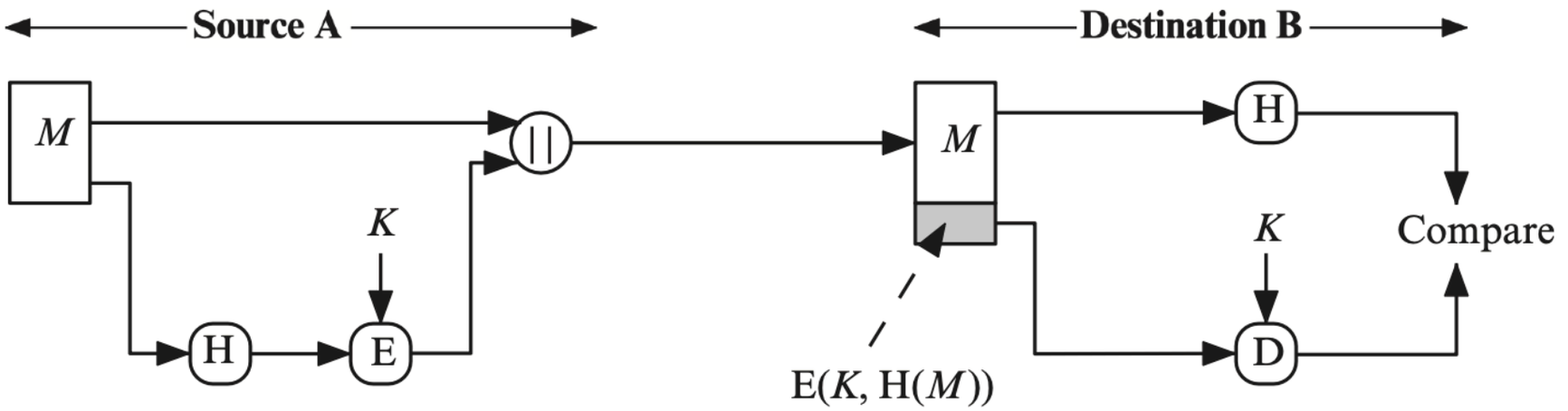
The message plus the hash code is encrypted using symmetric encryption.

- As A and B share the key, the message must have come from A and has not been altered.
- By encrypting the entire message plus the hash code, confidentiality is also provided.

Data Integrity

Message Authentication

Using hash code and symmetric encryption for message authentication



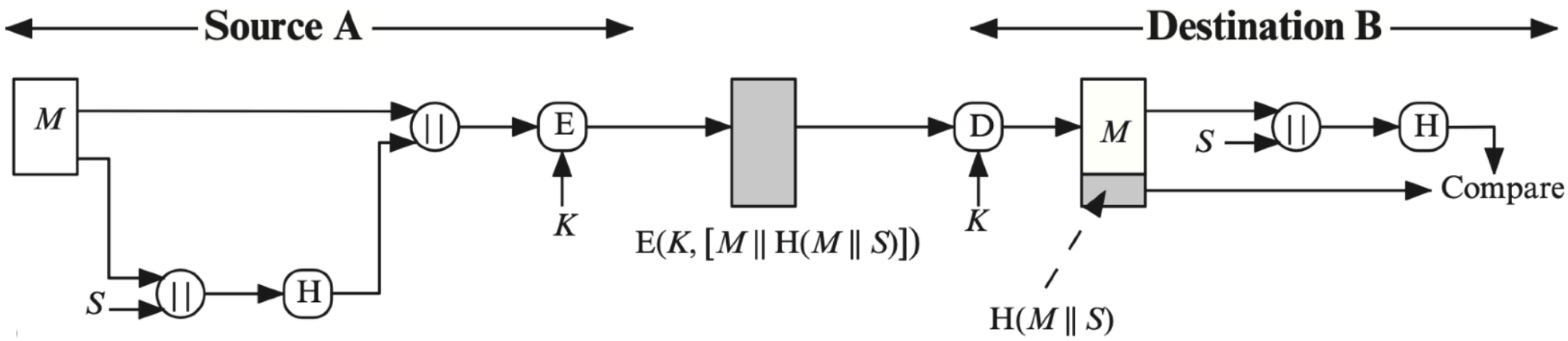
Only the hash code is encrypted using symmetric encryption.

- This reduces the processing burden.
- It does not provide confidentiality.

Data Integrity

Message Authentication

Using hash code and symmetric encryption for message authentication



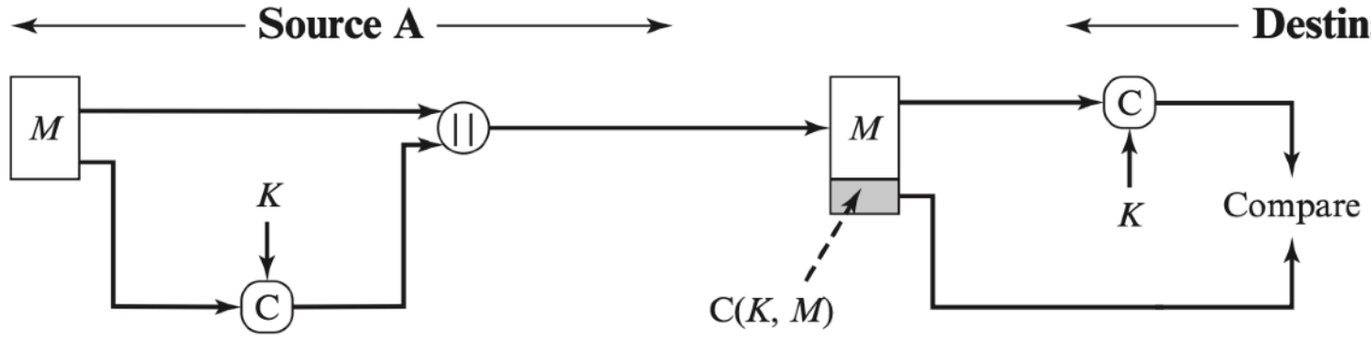
- It requires that two communicating parties share two different common secret values, **S** and **K**.
- The hash value is computed over the concatenation of **M** and **S**.

Data Integrity

Message Authentication Code - MAC

A keyed one-way function that generates a cryptographic fixed-length checksum appended to the message

Since K is shared only between A and B, the message origin must be A



MAC = C(K,M)

It provides authentication, but lacks message confidentiality

M = input message of arbitrary length

C = MAC function

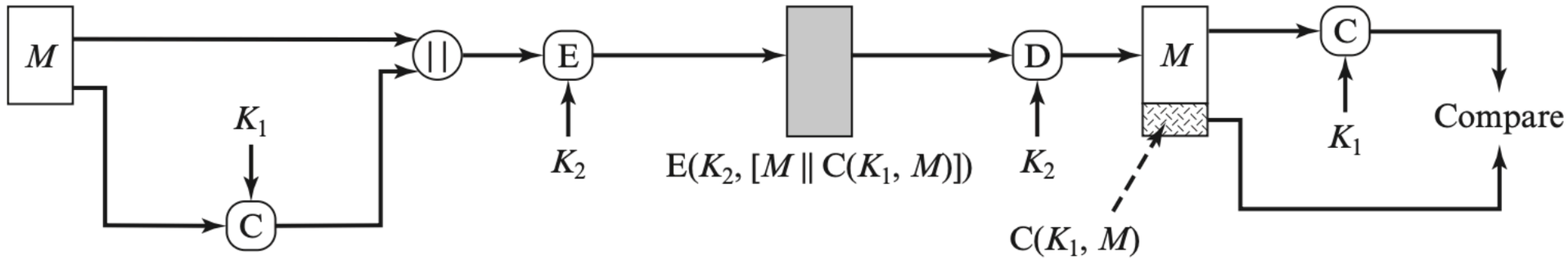
K = shared secret key

MAC or tag = message authentication code of fixed length

Data Integrity

Message Authentication Code - MAC

Message authentication and confidentiality

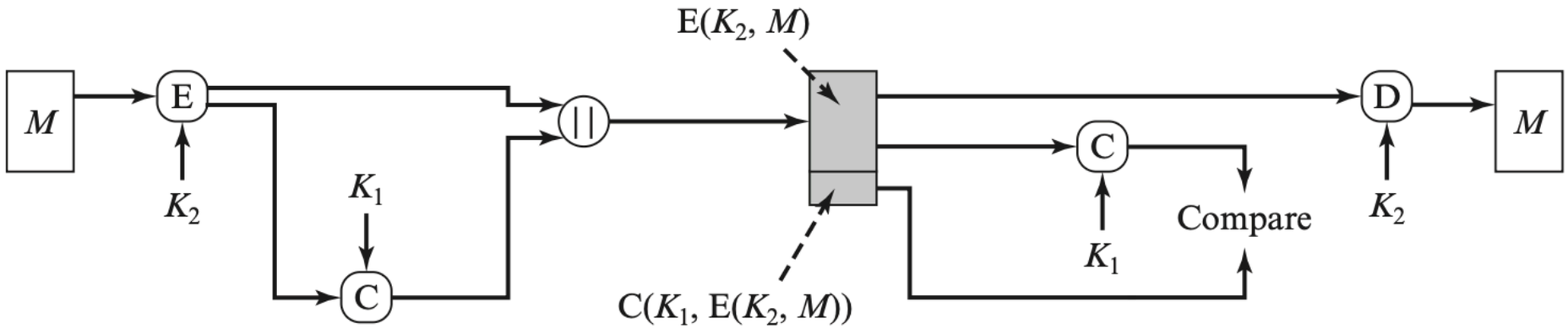


- It requires that two communicating parties share two different common secret values, K_1 and K_2.
- The authentication code is tied to the **plaintext**

Data Integrity

Message Authentication Code - MAC

Message authentication and confidentiality

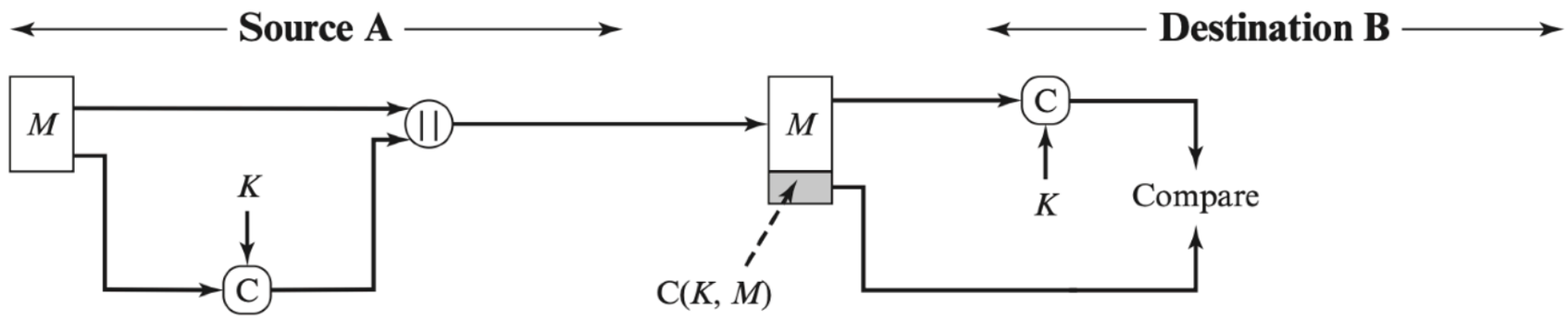


- It requires that two communicating parties share two different common secret values, K_1 and K_2.
- The authentication code is tied to the **ciphertext**

Data Integrity

Message Authentication Code - MAC

Security of MACs



- A brute-force attack on a MAC is more challenging than a brute-force attack on a hash function because it requires known message-tag pairs.
- The security of a MAC algorithm depends on the relative length of the `key` and the `tag` [`length(key) > length (tag)`]

Data Integrity

Hash-based MAC (*HMAC*)

Although hash functions were not designed for use as a MAC - they don't rely on a secret key - they are particularly interesting since:

1. Generally, they execute faster in software compared to symmetric block ciphers
2. Cryptographic hash function libraries are widely available

*Keyed-hashing for message authentication** is mandatory in multiple protocols (e.g., IPSec, TLS, SSH…), and is standardized through RFC 2104 and FIPS 198

* HMAC is also referred to as Keyed-Hashing for Message Authentication

Data Integrity

Hash-based MAC (*HMAC*)

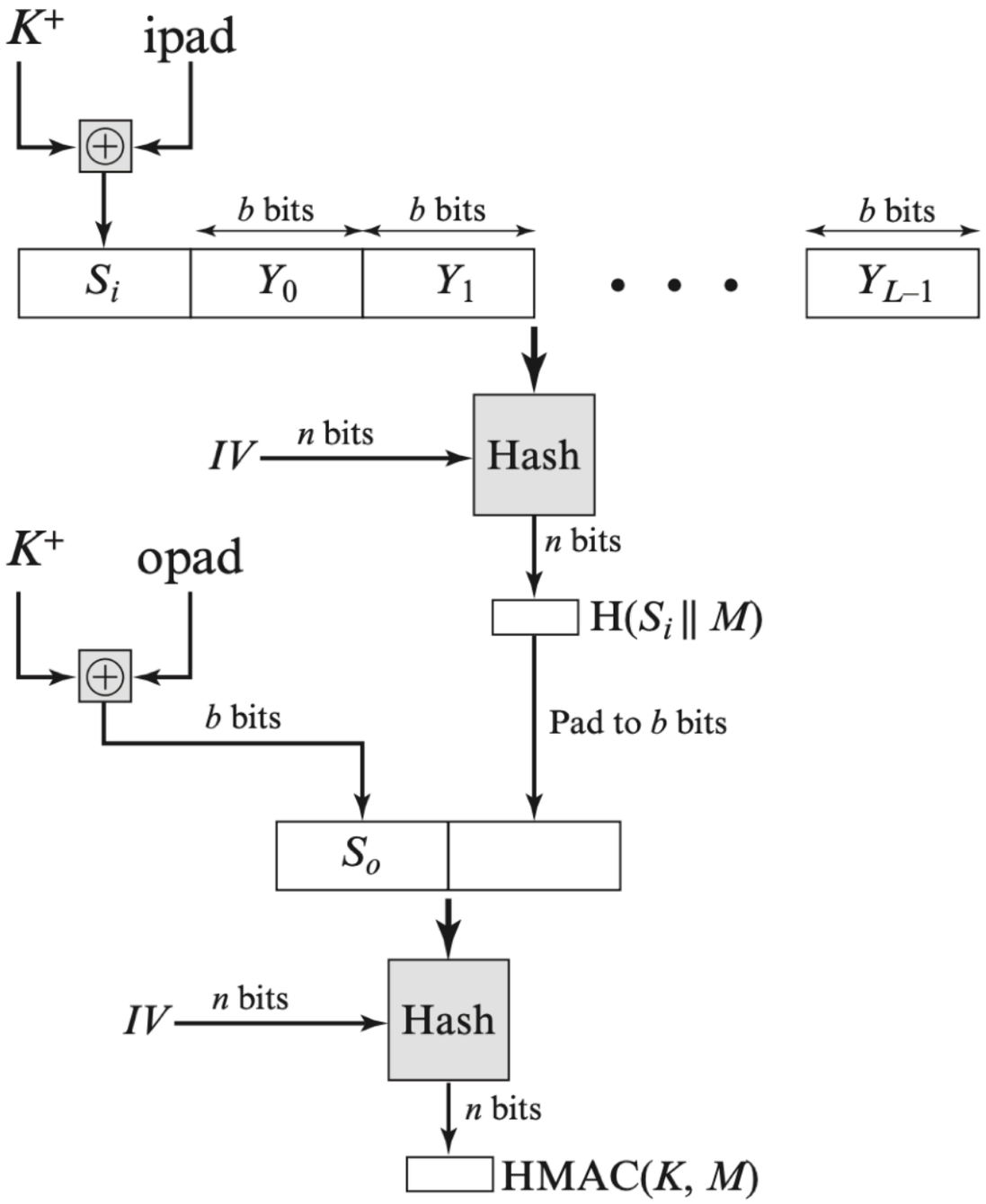
HMAC uses a modular design to…

- use existing cryptographic hash functions widely available in multiple code libraries
- allow easy replaceability in case faster or more secure functions are developed or required
- preserve the original performance of the hash function without adding computational overhead

Also, it inherits the security assumptions of the used cryptographic function

Data Integrity

Hash-based MAC (*HMAC*)



`H` - embedded hash function

`IV` - initial value input to the hash function

`M` - message input to HMAC

`Yi` - *ith* block of `M`

`L` - number of blocks in `M`

`b` - number of bits in a block

`n` - length of the hash digest

`K` - secret key

`K+` - `K` padded with zeros on the left, so `lenght(K+) == n`

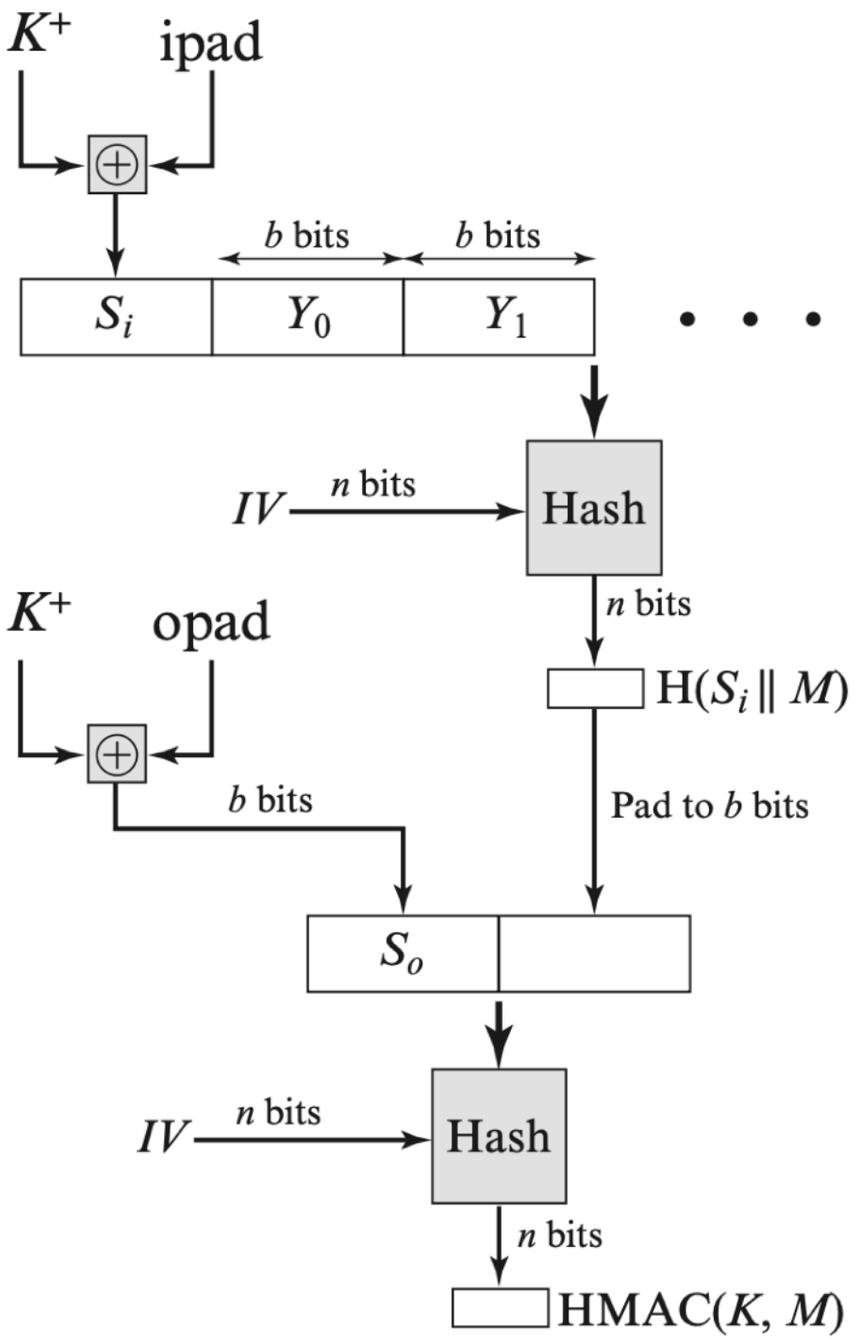
`ipad` - `36` in hexadecimal repeated b/8 times

`opad` - `5C` in hexadecimal repeated b/8 times

Source: Stallings, W. & Brown, L. Cryptography and Network Security: Principles and Practice

Data Integrity

Hash-based MAC (*HMAC*)



HMAC can be expressed as

HMAC(K, M)= H[(K⁺ ⊕ opad) } H[(K⁺ ⊕ ipad) } M]]

K⁺ ⊕ ipad
K⁺ ⊕ opad

They flip one-half of the bits of K

```
ipad = 00110110
opad = 01011100
```

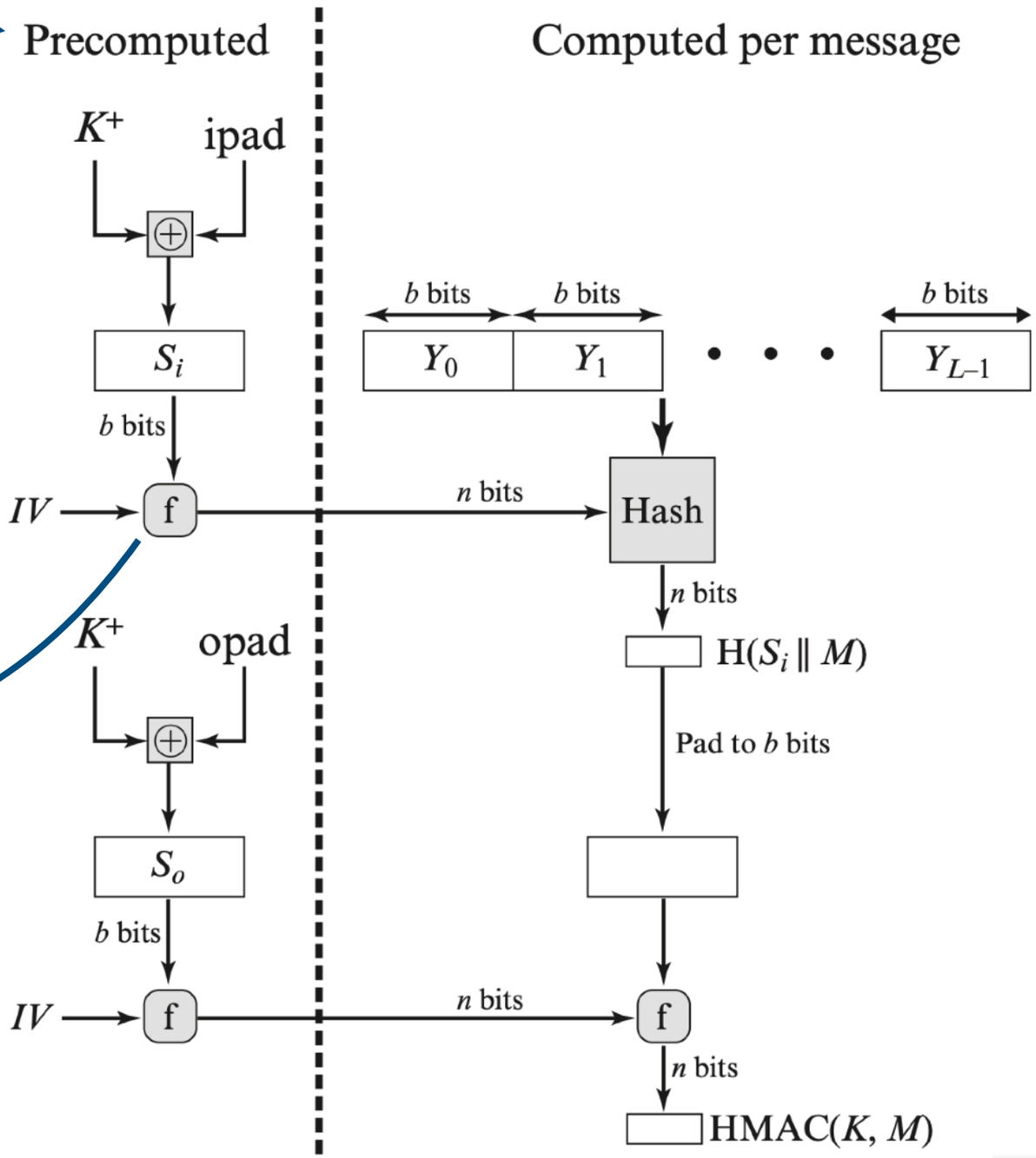
Source: Stallings, W. & Brown, L. Cryptography and Network Security: Principles and Practice

Data Integrity

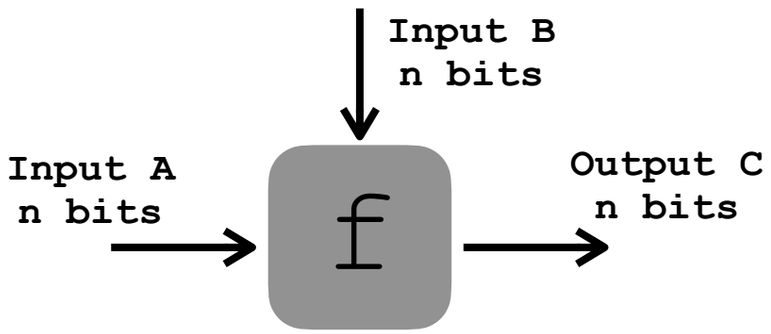
Hash-based MAC (*HMAC*)

A more efficient approach

It only needs to be recalculated when `K` is changed.



The compression function of `H`



Source: Stallings, W. & Brown, L. Cryptography and Network Security: Principles and Practice

Data Integrity

Authenticated Encryption

An encryption system that provides confidentiality and integrity
(due to the secrecy of the key, it also provides source authenticity)

Common approaches:

- Hashing followed by encryption (**H → E**)

$$E(K, (M||h))$$

- Authentication followed by encryption (**A → E**)

$$E(K_2, [M||MAC(K_1, M)])$$

- Encryption followed by authentication (**E → A**)

$$C = E(K_2(M)) || MAC(K_1, C)$$

- Independently encrypt and authenticate (**E + A**)

$$C = E(K_2(M)) \quad , \quad T = MAC(K_1, M)$$

They all require two operations:

1. Data encryption
2. Data authentication

Data Integrity

Authenticated Encryption

Recent approaches combine both operations

Symmetric block ciphers

- Galois/Counter Mode of Operation (GCM) → used in TLS 1.3
- Counter mode with CBC-MAC (CCM) → used in TLS 1.3
- Offset Codebook Mode (OCB)

Symmetric stream ciphers

- ChaCha20-Poly1305 → used in TLS 1.3
- AEGIS*

* All AEGIS variants can be used as stream ciphers (see external link to the draft)

Data Integrity

Authenticated Encryption

Galois/Counter Mode

Based on the CTR mode of operation, with the addition of a MAC that authenticates both the message and additional data that requires only authentication. It uses two main functions:

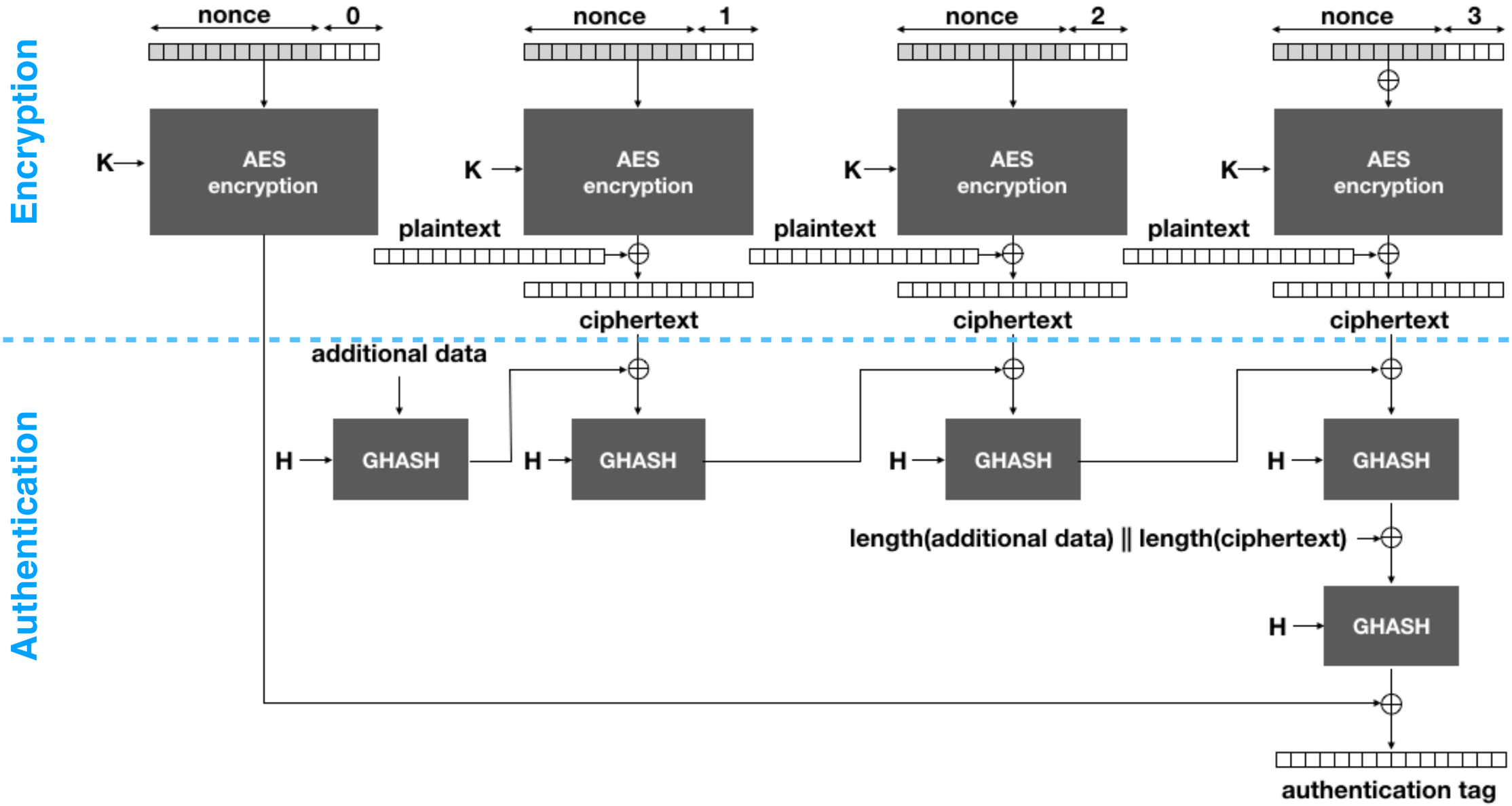
- GHASH - keyed hash function
- GCTR - the CTR mode in the Galois Field

- It also provides a mode that supplies the MAC only
- Requires a single secret key

Data Integrity

Authenticated Encryption

Galois/Counter Mode (example with AES)

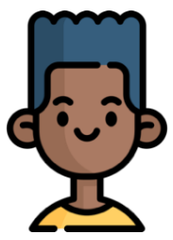


Data Integrity

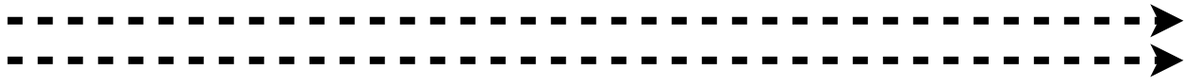
Authenticated Encryption

Which security properties are protected in this scheme?

What about non-repudiation?

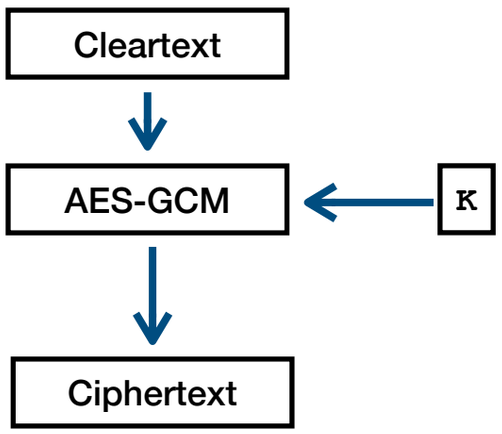


Bob

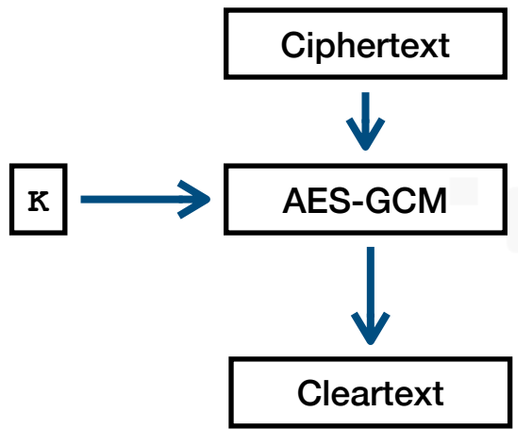




Alice







Data Integrity

Hands-on

Tamper Detection with Authenticated Encryption. Using AES-GCM or ChaCha20-Poly1305 (e.g., Python's cryptography or OpenSSL), encrypt a short message and verify that it decrypts correctly. Then flip one byte of the ciphertext (e.g., XOR with 0x01) and try decrypting again. What happens, and is any plaintext returned? For comparison, do the same with unauthenticated AES-CTR and observe how the decrypted output changes.



Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

University of Minho
School of Engineering